

When Does LLM-Serving Scheduler Adaptation Matter?

Action Opportunity and Causal Headroom in Production-Derived Replay

Soroush Vahidi¹

New Jersey Institute of Technology, Newark, NJ, USA
sv96@njit.edu

Abstract. Adaptive scheduling is often justified by heterogeneous workloads and competing serving objectives. We study a more basic question: before adding scheduler complexity, does a strong default actually have a different useful executable action to take? We define canonical action opportunity as the prevalence $P(D)$ of states where a fixed six-policy portfolio produces an executable action different from the single best scheduler (SBS), and separate it from default-relative one-step causal headroom. Faithful replay of 60 windows from Azure 2023 code, Azure 2023 conversation, and BurstGPT was action-null under abundant resources: the three workloads produced 0/99,992, 0/461,985, and 0/440,461 canonical disagreements, respectively. A frozen one-axis pressure map then showed that KV-capacity and active-sequence constraints, rather than arrival scaling through $8\times$, created disagreement. On untouched fresh windows, the confirmatory latency study found 720 disagreement states, 590 with a beneficial alternative, $P(B_{\text{LAT}} \mid D) = 0.8194$, and mean oracle headroom of 1.996 ms with a clustered 95% confidence interval $[0.185, 3.497]$ ms. Terminal ANWG saturated at one and therefore concealed latency differences in the earlier pressure study. A bounded local vLLM validation supported the pressure/action mechanism at the observable queue and scheduler-trace levels, but did not implement a controlled one-step intervention and did not reproduce the simulator’s qualitative reversal. Thus adaptive scheduling is conditional: resource pressure must first create action opportunity, and opportunity must then be tested for causal value; neither step establishes a learned deployable selector.

Keywords: LLM serving · scheduling · resource pressure · causal headroom · production-derived workloads

1 Introduction

Modern LLM-serving systems adapt to prompt length, decode demand, KV-cache residency, queueing, deadlines, and changing load. This is necessary in many regimes, but it also creates a practical evaluation problem. A policy may

look different in its ranking score while producing the same executable batch, admission, or request order as a strong default. Conversely, a different action may exist without improving the operational metric that motivated adaptation.

We therefore ask a prior question to scheduler design: *when is there actually a different useful action to take?* This question complements systems that improve routing, caching, preemption, batching, or resource configuration. Serve-Gen demonstrates why production-shaped workloads matter for serving evaluation [17]; LMetric demonstrates that simple KV-aware and load-aware routing can materially improve latency on real workloads [22]. We do not replace these systems. We measure a different layer of the argument: the transition from resource pressure to canonical executable choice, and then from choice to local causal value.

The paper follows the decomposition

$$\begin{aligned} \text{resource pressure} &\longrightarrow P(D) \longrightarrow P(B_{\text{LAT}} \mid D) \\ &\longrightarrow \text{headroom magnitude} \longrightarrow \text{predictability and deployment.} \end{aligned} \tag{1}$$

Here D means that at least one canonical alternative action differs from SBS, while B_{LAT} means that at least one such action improves one-step SBS-relative mean latency. The last arrow is deliberately left unclaimed: a causal opportunity is not a learned controller.

We answer four research questions.

- RQ1 – Action opportunity.** Under faithful production-derived replay, how often do SBS and the P6 portfolio produce genuinely different canonical executable actions?
- RQ2 – Resource-regime transition.** How does resource pressure govern the transition from scheduler agreement to disagreement?
- RQ3 – Causal headroom.** When disagreement exists, how often is SBS locally suboptimal for operational latency, and how large is one-step oracle headroom?
- RQ4 – Objective and system validity.** How sensitive are conclusions to the objective, and does the pressure/action mechanism appear in a bounded real-vLLM deployment?

The central result is conditional rather than universal. Native replay is action-null under the abundant-resource baseline. Concrete KV and active sequence pressure creates executable alternatives. Among selected untouched fresh disagreement states, most contain positive local mean-latency headroom, but the mean is modest and the opportunity prevalence can be tiny. The real-system result supports the structural pressure/action mechanism only partially, and we make no claim that a selector has learned to exploit it.

1.1 Contributions

This paper makes four contributions:

1. An action-opportunity methodology that distinguishes canonical executable disagreement from policy-score diversity or whole-policy ranking differences.
2. A production-derived regime characterization showing action collapse under native resource abundance and disagreement onset under concrete KV or active-sequence pressure.
3. A fresh default-relative one-step causal analysis that confirms positive mean-latency headroom on untouched disagreement states while retaining the full-state opportunity denominator.
4. An objective and systems qualification: terminal ANWG can saturate while latency changes, and bounded vLLM evidence supports R1/R2 pressure and action differentiation without supplying R3 causal intervention.

2 Related Work and Positioning

LLM-serving research spans model execution, prefill/decode scheduling, resource management, routing, caching, and cluster control. Orca and vLLM established iteration-level serving and paged KV management [21, 4]. Sarathi-Serve, Dist-Serve, Splitwise, and FastServe study chunking, phase separation, and preemptive scheduling [1, 23, 8, 16]. Llumnix and SOLA adapt scheduling or state-aware service to dynamic conditions [14, 3]; Libra studies flexible request partitioning [11]; Mooncake and Strata emphasize KV-centric or hierarchical context caching [9, 18]; MorphServe changes model precision and KV capacity under pressure [13]; and DynamoLLM reconfigures cluster resources for performance and energy [12]. BurstGPT and Azure traces provide important workload evidence [15, 6]; the recent longitudinal study by Nixon et al. further emphasizes workload evolution, caching, and load-balancing over a year of production traffic [7].

ServeGen and OpenTela narrow any generic claim that production realism or operational trace infrastructure is itself novel [17, 20]. LMetric is especially close: it combines a KV-aware indicator with an instance-load indicator, reports production-style workloads, and derives conditions under which its multiplication can fail [22]. These works answer how to build or evaluate better serving mechanisms. Our question is complementary: in a fixed workload/resource regime, do the default and alternatives expose different canonical actions at all, and does forcing one alternative action have default-relative causal value?

Table 1 records the scope honestly. A “partial” entry means that a work contains related evidence but does not use the paper’s canonical action or one-step estimand.

Our use of SBS/VBS is rooted in algorithm portfolios and selection [10, 2, 19, 5]. The novelty claim is not the envelope itself, nor counterfactual reasoning in general. It is the narrow serving characterization in Eq. 1.

Table 1. Closest-work differentiation. We do not claim that other systems omit pressure, production traces, or adaptive mechanisms; the distinction is the action-level and default-relative causal measurement question.

Work	Prod.-derived workloads	Pressure sweep	Proposes system	Canonical disagreement	One-step default intervention	Real system
ServeGen	Yes	No	Generator	No	No	Production workload study
LMetric	Yes	Implicit	Yes	No	No	Yes
OpenTela	Operational traces	No	Control plane	No	No	Yes
Llumnix	Selected regimes	Partial	Yes	No	No	System evaluation
SOLA						
Mooncake	/ Cache/resource regimes	Partial	Yes	No	No	Yes / production
Strata						
This work	Yes	Yes	No	Yes, $P(D)$	Yes, simulator only	Bounded partial

3 Methods

3.1 Evidence hierarchy and workload inventory

We report four evidence tiers. Tier 1 is native production-trace replay. Tier 2 is production-derived aggregate or window validation. Tier 3 is a controlled transformation of a real trace, such as one-axis resource caps. Tier 4 is synthetic or mechanistic workload construction. RQ1 uses Tier 1; RQ2 uses Tiers 2–3; fresh causal headroom uses a preregistered Tier-3 intervention over untouched windows; the real-vLLM result is a bounded system validation rather than a claim of production deployment.

The faithful corpus contains 20 windows each from Azure 2023 code, Azure 2023 conversation, and BurstGPT. Source order and token-length/arrival structure are retained. SLO and class overlays are deterministic simulator inputs; future actual output lengths are hidden from policies. The fresh confirmatory campaign uses 20 untouched windows per workload family for support mapping, with causal labeling limited to the frozen selected regimes. BurstGPT did not contribute a selected fresh causal disagreement regime, which is retained as an explicit heterogeneity limitation.

The historical P6 portfolio contains full prefill, small-chunk prefill, estimated-service-first (ESTF), weighted fair sharing (WFS), least-laxity first (LLF), and a KV-constrained policy. P6 is a fixed portfolio for a default-relative characterization, not an exhaustive set of 2026 schedulers.

3.2 Canonical action opportunity

Let $\mathcal{A}_6(s)$ be the set of canonical executable actions produced by the six portfolio policies at state s , after action deduplication. Let $a_{\text{SBS}}(s)$ be SBS’s action. We define

$$D(s) = \mathbf{1} [\exists a \in \mathcal{A}_6(s) : a \neq a_{\text{SBS}}(s)]. \quad (2)$$

The opportunity prevalence is $P(D) = \sum_s D(s) / |\mathcal{S}_{\text{all}}|$, where the denominator is every valid SBS decision state, not only states where policy scores differ. A policy ranking change that produces the same admission, batch, and request order is action collapse, not opportunity.

This definition matters operationally. Whole-policy comparisons can show a latency or goodput difference accumulated over many decisions, but cannot say

whether an alternative existed at a particular default state. Conversely, action diversity alone says nothing about usefulness.

3.3 One-step causal headroom

For every disagreement state s and canonical alternative a , we clone the state, force exactly one action, and return to SBS for the continuation. Denote the resulting continuation value by $Q_{\text{SBS}}(s, a)$. This is a local counterfactual, not selector performance.

For the fresh primary latency endpoint, continuation mean latency includes active, queued, and future requests belonging to the continuation, excluding requests completed before s . We define

$$A_{\text{LAT}}(s, a) = L_{\text{SBS}}(s) - L_{\text{CF}}(s, a), \quad H_{\text{LAT}}(s) = \max \left(0, \max_a A_{\text{LAT}}(s, a) \right), \quad (3)$$

where positive values reduce mean latency. The beneficial-state indicator is $B_{\text{LAT}}(s) = 1$ iff $\max_a A_{\text{LAT}}(s, a) > 0$. We report $P(B_{\text{LAT}} \mid D)$ and mean H_{LAT} separately from $P(D)$. The conceptual aggregate opportunity is therefore

$$P(D) \times P(B_{\text{LAT}} \mid D) \times \text{effect magnitude}, \quad (4)$$

not an exact deployable-performance equation. A high conditional benefit can coexist with a tiny overall opportunity.

The confirmatory mean uses a faithful-window clustered bootstrap with 2,000 replicates and seed 20260920. Its verdict requires positive mean headroom and a clustered 95% lower endpoint above zero. State counts are not treated as independent samples.

3.4 Objective audit

The historical objective is arrival-normalized weighted goodput,

$$\text{ANWG} = \frac{\sum_i w_i \mathbf{1}[i \text{ completes by its SLO}]}{\sum_i w_i}. \quad (5)$$

Phase-D V1’s all-terminal-ANWG result is retained as an objective audit, not as fresh latency evidence. The latency study uses Eq. 3; its mean-latency result is confirmatory only for the fresh untouched causal universe.

4 Results: Native Action Opportunity

Under the frozen abundant-resource baseline, all three production-derived workloads were canonically action-null. Azure code had 0 disagreements in 99,992 SBS decision states; Azure conversation had 0/461,985; and BurstGPT had 0/440,461. The corresponding maximum KV utilizations were 0.002716, 0.001728, and 0.003802, with no active-cap binding states. Policy-ranking proxy differences existed in a small number of states, but canonical actions collapsed. This is not evidence that adaptive scheduling never matters; it is evidence that this resource regime offered no different executable P6 action.

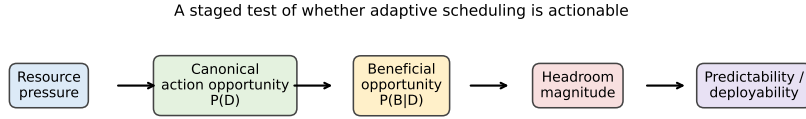


Fig. 1. The paper’s evidence chain. A predictor is downstream of both action opportunity and causal value; it is not assumed by the characterization.

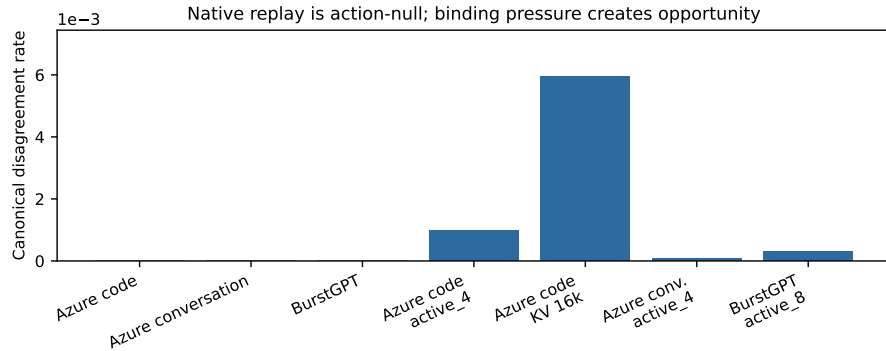


Fig. 2. Native production-derived replay is action-null. Selected pressure conditions create nonzero canonical disagreement, with rates depending on workload and pressure axis.

5 Results: Resource-Regime Transition

The Phase-B V2 map evaluated 1,080 one-axis conditions: 20 windows across three workload families, with active-sequence, KV-capacity, and arrival pressure axes. There were 926 valid unconstrained, 38 partially constrained, 96 strongly constrained, and 20 horizon-invalid conditions. Across valid conditions, 11,328 canonical disagreement states were observed.

Arrival scaling through $8\times$ remained action-null for all three workload families. Active-sequence pressure created disagreement at active-cap 8 or 4 depending on workload; KV pressure created disagreement at the first binding or near-binding capacities. For Azure code, the first active-cap disagreement appeared at active_8 and became sustained at active_4. KV disagreement began at kv_16000. Azure conversation first disagreed at active_4 and showed KV disagreement at kv_16000; BurstGPT showed active disagreement at active_16, sustained at active_8, and KV disagreement at kv_16000.

The transition is the main regime result: pressure does not automatically create useful diversity, but concrete resource binding can turn a collapsed action set into a nontrivial one.

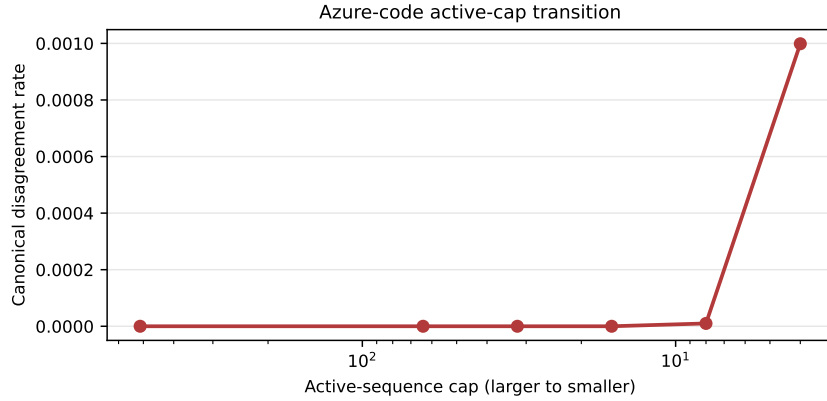


Fig. 3. Azure-code active-sequence transition. Agreement persists until the active cap binds the trace, after which canonical disagreement increases.

Table 2. Pressure-to-disagreement transition summary. “Sustained” means at least two windows with disagreement.

Workload	Axis	First binding	First disagreement	Sustained
Azure code	Active cap	active_8	active_8	active_4
Azure code	KV cap	kv_16000	kv_16000	kv_16000
Azure conversation	Active cap	active_4	active_4	active_4
Azure conversation	KV cap	kv_8000	kv_16000	kv_8000
BurstGPT	Active cap	active_16	active_16	active_8
BurstGPT	KV cap	kv_16000	kv_16000	kv_16000

6 Results: Fresh Support and Causal Latency Headroom

The fresh support mapping used untouched windows and the frozen pressure protocol. The causal labeling plan selected five Azure code/conversation active/KV regimes and 720 disagreement states. BurstGPT contributed no selected causal regime, an important limitation rather than a missing result.

The confirmatory intervention found 590 beneficial states, giving

$$P(B_{\text{LAT}} | D) = 590/720 = 0.8194444444.$$

Mean oracle latency headroom was 0.001995791 s (1.996 ms), with a clustered 95% CI of [0.000184589, 0.003497418] s ([0.185, 3.497] ms). The frozen verdict was **LATENCY_HEADROOM_CONFIRMED**. This is statistically supported but operationally modest. It is not the claim that 81.9% of all scheduler decisions can be improved: $P(D)$ is often below one percent in the selected regimes and the conditional statistic is defined only on disagreement states.

6.1 Opportunity decomposition

The fresh results make the distinction between prevalence and value concrete. Azure-code kv_16000 has the largest selected $P(D)$ and the largest mean head-

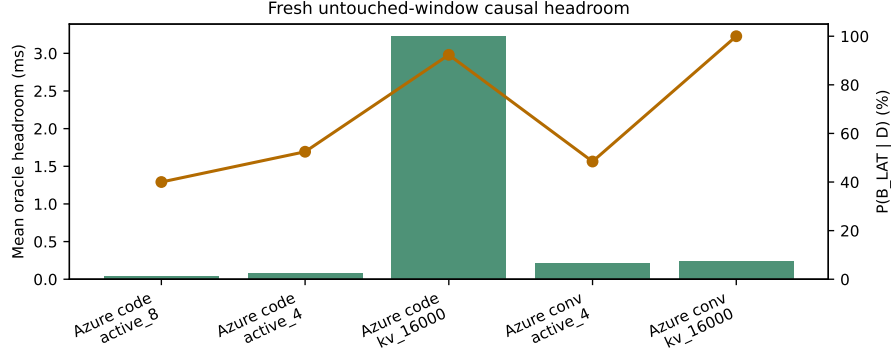


Fig. 4. Fresh causal result by selected workload/regime. Bars show mean oracle headroom in milliseconds; the line shows $P(B_{LAT} | D)$.

Table 3. Fresh causal characterization. $P(D)$ uses all valid SBS decision states; $P(B_{LAT} | D)$ uses only the disagreement states.

Workload/regime	States	$P(D)$	$P(B_{LAT} D)$	Mean H_{LAT} (ms)	Windows
Azure code, active_8	10	0.000106	0.400	0.033	2
Azure code, active_4	122	0.001290	0.525	0.077	15
Azure code, kv_16000	431	0.004561	0.923	3.227	14
Azure conversation, active_4	64	0.000108	0.484	0.217	17
Azure conversation, kv_16000	93	0.000157	1.000	0.241	1
All selected fresh states	720	—	0.819	1.996	26 clusters

room; active-cap regimes have much smaller $P(D)$ and correspondingly smaller aggregate opportunity even when some disagreement states are beneficial. A practitioner should therefore measure all three factors in Eq. 4: whether a different action exists, whether it helps conditional on existing, and whether its effect is large enough to matter relative to system noise and control overhead.

7 Objective Sensitivity: ANWG Saturation

The earlier Phase-D V1 pressure study completed with causal integrity and complete branch coverage, but all terminal ANWG values were one and all ANWG advantages were zero. An identifiability audit found no implementation bug: requests completed within deadlines and the objective saturated. Simple sensitivity cases confirmed that ANWG distinguishes missed deadlines or unfinished work, but not different latency or queueing when all requests still meet a loose deadline.

Most counterfactual branches nevertheless changed latency. Thus zero ANWG headroom does not imply zero latency headroom. We do not use the old Phase-D post-hoc latency analysis as confirmatory evidence. The fresh untouched study

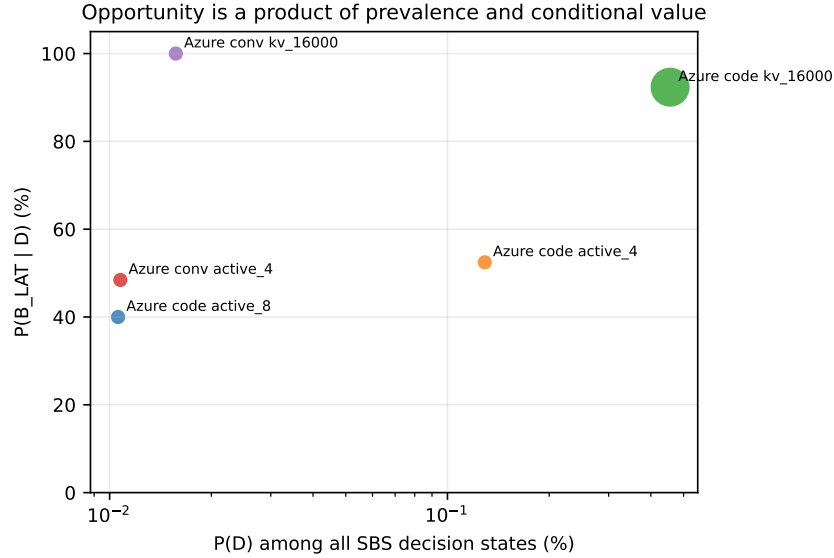


Fig. 5. Practitioner regime map. Conditional benefit can be high while opportunity prevalence is small; point size is proportional to mean headroom.

in Section 6 is the confirmatory latency result, with its own pre-frozen primary endpoint and clustered bootstrap.

This is objective sensitivity, not evidence that ANWG is invalid. ANWG is an appropriate deadline-goodput objective for one question; mean latency is a different operational objective. A serving paper should not promote whichever metric yields the more favorable conclusion.

8 Bounded Real-vLLM Validation

The bounded systems validation used one RTX 5060 Ti, vLLM 0.27.1, Qwen2.5-0.5B-Instruct, and deterministic token-shape replay. It completed 40 FULL-versus-CHUNKED runs and a 20-run native chunked-prefill budget probe. Queueing and scheduler traces were retained. The validation is classified **PARTIAL_SUPPORT**: R1 pressure validation and R2 observable action differentiation passed; R3 exact one-step controlled intervention was not available.

The native budget probe held chunked prefill fixed and varied the token budget. T512 produced 1,680 scheduler steps, 165 mixed prefill/decode steps, and 194 partial-prefill items. T4096 produced 1,536, 55, and 9. Waiting queues reached 7 and 5, respectively, while running sequences reached 4 in both cases. The low-late regime showed a 30.6 ms late-TTFT difference and a 16.3 ms prompt-heavy E2E tradeoff; the high-late regime showed a smaller late-TTFT difference and a 23.3 ms prompt-heavy E2E tradeoff.

Table 4. Simulator-to-real-system correspondence.

Property	Simulator/replay	Real vLLM
Low pressure	Canonical collapse	Low queue baseline
Resource binding	KV/active caps induce D	Queue and active-sequence pressure observed
Action difference	SBS/P6 canonical action	Native token-budget trace signature
Latency consequence	One-step SBS-relative	Whole-run budget tradeoff
Exact intervention	Available	Not available

The direct FULL-versus-CHUNKED comparison did not reproduce the simulator’s qualitative reversal. KV utilization and preemption metrics were unavailable in the native probe because its metric parser returned empty dictionaries. The real result therefore supports the narrower statement that concrete serving pressure can alter observable scheduling behavior and latency. It is not real-system causal confirmation of Eq. 3.

9 Discussion

9.1 When adaptation is justified

The evidence does not support an anti-adaptive-scheduling conclusion. It supports a staging rule. Under abundant resources, a complex portfolio may collapse to the same action as SBS, making adaptation overhead difficult to justify. Under binding KV or active-sequence pressure, executable alternatives can appear. Some fresh disagreement states contain positive latency headroom, but opportunity prevalence and effect magnitude remain regime-specific.

This matters because scheduler diversity is not scheduler value. A system that produces many distinct actions may still optimize the wrong objective; a system with high conditional benefit may have too few disagreement states to justify a router. The decomposition in Eq. 4 is a practical diagnostic, not a promise of deployment gain.

9.2 Relation to modern adaptive systems

LMetric, SOLA, Llumnix, MorphServe, Strata, Libra, FastServe, and related systems answer how to adapt serving behavior under dynamic resource and SLO conditions. ServeGen, BurstGPT, Azure traces, OpenTela, and longitudinal trace studies answer how to obtain or operate on realistic workload evidence. Our measurement asks when adaptation is unnecessary because canonical actions collapse, and when a local alternative is causally useful relative to a strong default. The questions are complementary; the present P6 study does not claim to replace or exhaust modern schedulers.

9.3 Limitations

The P6 portfolio is not the universe of 2026 schedulers. The study uses three principal production-derived trace families and a simulator/replay semantic model. BurstGPT did not contribute a selected fresh causal regime. Fresh headroom is a mean-latency one-step oracle result, not a learned selector or production improvement; its mean effect is modest. The real-vLLM validation uses one model, GPU, and mechanism family, uses token-shape replay rather than exact private prompts, and supports R1/R2 only. It has no exact one-step real-system intervention, and the simulator-to-vLLM action mapping is imperfect. ANWG saturation limits conclusions under that objective. Finally, we make no claim of learned predictability, cross-distribution generalization, universal scheduler superiority, or acceptance probability.

10 Reproducibility and Artifact Scope

The public artifact records trace provenance, frozen manifests, support grids, fresh non-overlap proofs, causal state and action manifests, source hashes, bootstrap seed 20260920, and the real-vLLM environment. The principal result artifacts are:

- Phase A native replay: `phase_a_v1`
- Phase B pressure map: `phase_b_v2`
- Fresh causal latency: `fresh_production_latency_headroom_confirmatory_v1`
- Real vLLM package: `real_vllm_pressure_action_validation_v1`

Historical selector and joint-240 artifacts remain available but are not treated as evidence of learned deployment readiness.

11 Conclusion

Adaptive LLM-serving scheduling should be evaluated in stages. Faithful production-derived replay can be action-null under abundant resources; resource pressure can create canonical executable alternatives; and fresh one-step replay can show that some alternatives contain positive local mean-latency headroom. In this study, the fresh confirmation found 590 beneficial states among 720 disagreement states, with 1.996 ms mean oracle headroom and a positive clustered confidence interval. The result is conditional and modest: $P(B_{\text{LAT}} \mid D)$ is not $P(B_{\text{LAT}})$, and neither is a deployable selector.

Terminal ANWG saturation demonstrates why objective choice matters, while bounded vLLM evidence provides partial real-system support for pressure and observable action differentiation without claiming real causal headroom. The shortest path forward is final manuscript and artifact audit, not another selector or broad trace experiment.

Acknowledgements The author thanks Professor Ioannis Koutis for guidance and support.

Data and Code Availability Code, workload generators, configurations, and derived artifacts are publicly available at <https://github.com/SoroushVahidi/llm-serving-heuristic-evolution>. Raw third-party traces are obtained from their upstream sources; derived windows and manifests are documented in the repository.

Generative AI Use The author used language models and coding assistants for organization, language refinement, literature-search planning, software development, and workflow support. These tools were not treated as scientific evidence. The author independently verified designs, results, interpretations, citations, and final content.

Declaration of Competing Interest The author declares no competing interests relevant to this work.

References

1. Agrawal, A., Kedia, N., Panwar, A., Mohan, J., Kwatra, N., Gulavani, B.S., Tumanov, A., Ramjee, R.: Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve. In: 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24). pp. 117–134. USENIX Association (2024)
2. Gomes, C.P., Selman, B.: Algorithm portfolios. *Artificial Intelligence* **126**(1–2), 43–62 (2001)
3. Hong, K., Li, X., Chen, L., Mao, Q., Dai, G., Ning, X., Yan, S., Liang, Y., Wang, Y.: SOLA: Optimizing SLO attainment for large language model serving with state-aware scheduling. In: *Proceedings of Machine Learning and Systems*. vol. 7 (2025)
4. Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C.H., Gonzalez, J.E., Zhang, H., Stoica, I.: Efficient memory management for large language model serving with PagedAttention. In: *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP 2023)*. pp. 611–626. ACM (2023)
5. Lindauer, M., Hoos, H.H., Hutter, F., Schaub, T.: AutoFolio: An automatically configured algorithm selector. *Journal of Artificial Intelligence Research* **53**, 745–778 (2015)
6. Microsoft Azure: Azure LLM inference trace 2023. <https://github.com/Azure/AzurePublicDataset/blob/master/AzureLLMInferenceDataset2023.md> (2023), public trace dataset accompanying Splitwise
7. Nixon, W., Durbin, J., Standhartinger, F., Gunawi, H.S., Yang, J.: A year in LLM serving: Workload evolution, caching and load-balancing. *arXiv preprint arXiv:2608.13573* (2026)
8. Patel, P., Choukse, E., Zhang, C., Shah, A., Goiri, I., Maleki, S., Bianchini, R.: Splitwise: Efficient generative LLM inference using phase splitting. In: *Proceedings of the 51st Annual International Symposium on Computer Architecture*. IEEE Computer Society (2024). <https://doi.org/10.1109/ISCA59077.2024.00019>
9. Qin, R., Li, Z., He, W., Cui, J., Ren, F., Zhang, M., Wu, Y., Zheng, W., Xu, X.: Mooncake: Trading more storage for less computation – a KVCache-centric architecture for serving LLM chatbot. In: *23rd USENIX Conference on File and Storage Technologies (FAST 25)*. pp. 155–170. USENIX Association (2025)
10. Rice, J.R.: The algorithm selection problem. *Advances in Computers* **15**, 65–118 (1976)

11. Ruan, C., Chen, Y., Tian, D., Shi, Y., Wu, Y., Li, J., Li, C.: Libra: Flexible request partitioning and scheduling for serving unbalanced and dynamic LLM workloads. In: 23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26). USENIX Association (2026)
12. Stojkovic, J., Zhang, C., Goiri, I., Torrellas, J., Choukse, E.: DynamoLLM: Designing LLM inference clusters for performance and energy efficiency. In: Proceedings of the 2025 IEEE International Symposium on High Performance Computer Architecture (HPCA 25). pp. 1348–1362. IEEE (2025). <https://doi.org/10.1109/HPCA61900.2025.00102>
13. Su, Z., Lan, T., Wang, Z., Yang, J., Cheng, Y.: Efficient and workload-aware LLM serving via runtime layer swapping and KV cache resizing. arXiv preprint arXiv:2506.02006 (2025)
14. Sun, B., Huang, Z., Zhao, H., Xiao, W., Zhang, X., Li, Y., Lin, W.: Llumnix: Dynamic scheduling for large language model serving. In: 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24). pp. 173–191. USENIX Association (2024)
15. Wang, Y., Chen, Y., Li, Z., Kang, X., Fang, Y., Zhou, Y., Zheng, Y., Tang, Z., He, X., Guo, R., Wang, X., Wang, Q., Zhou, A.C., Chu, X.: BurstGPT: A real-world workload dataset to optimize LLM serving systems. In: Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining. pp. 5831–5841. ACM (2025). <https://doi.org/10.1145/3711896.3737413>
16. Wu, B., Zhong, Y., Zhang, Z., Liu, S., Liu, F., Sun, Y., Huang, G., Liu, X., Jin, X.: FastServe: Iteration-Level preemptive scheduling for large language model inference. In: 23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26). pp. 57–74. USENIX Association (2026)
17. Xiang, Y., Li, X., Qian, K., Yu, W., Zhai, E., Jin, X.: ServeGen: Workload characterization and generation of large language model serving in production. In: Proceedings of the 21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 26). USENIX Association (2026), arXiv:2505.09999
18. Xie, Z., Xu, Z., Zhao, M., An, Y., Mailthody, V.S., Mahlke, S., Garland, M., Kozyrakis, C.: Strata: Hierarchical context caching for long context language model serving. arXiv preprint arXiv:2508.18572 (2025)
19. Xu, L., Hutter, F., Hoos, H.H., Leyton-Brown, K.: SATzilla: Portfolio-based algorithm selection for SAT. *Journal of Artificial Intelligence Research* **32**, 565–606 (2008)
20. Yao, X., Jiang, Y., Badanin, I., Hu, Q., Smith, R.M., Yuan, B., Schlag, I., Yoneki, E., Klimovic, A.: OpenTela: Unifying decentralized computing resources for heterogeneous LLM serving. In: 20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26). pp. 1821–1838. USENIX Association (2026)
21. Yu, G.I., Jeong, J.S., Kim, G.W., Kim, S., Chun, B.G.: Orca: A distributed serving system for transformer-based generative models. In: 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22). pp. 521–538. USENIX Association (2022)
22. Zhang, D., Han, J., Zhang, K., Wei, X., Shen, S., Fang, C., Yu, W., Zhou, J., Chen, R.: LMetric: Simple is better – multiplication may be all you need for LLM request scheduling. arXiv preprint arXiv:2603.15202 (2026)
23. Zhong, Y., Liu, S., Chen, J., Hu, J., Zhu, Y., Liu, X., Jin, X., Zhang, H.: Dist-Serve: Disaggregating prefill and decoding for goodput-optimized large language model serving. In: 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24). pp. 193–210. USENIX Association (2024)